

Module 10: TypeScript Design Patterns & Best Practices

1. Introduction to Design Patterns in TypeScript

Design patterns are reusable solutions to common software design problems.

2. Creational Design Patterns

Singleton Pattern

Ensures that a class has only one instance.

typescript

```
class Singleton {  
  private static instance: Singleton;  
  
  private constructor() {}  
  
  public static getInstance(): Singleton {  
    if (!Singleton.instance) {  
      Singleton.instance = new Singleton();  
    }  
    return Singleton.instance;  
  }  
}
```

```
const instance1 = Singleton.getInstance();  
const instance2 = Singleton.getInstance();  
console.log(instance1 === instance2); // true
```

Factory Pattern

Creates objects without specifying the exact class.

typescript

```
class Car {  
  constructor(public model: string, public price: number) {}  
}
```

```
class CarFactory {
```

```
static createCar(type: string): Car {  
  if (type === "sedan") return new Car("Sedan", 20000);  
  if (type === "suv") return new Car("SUV", 30000);  
  throw new Error("Unknown car type");  
}  
}
```

```
const myCar = CarFactory.createCar("sedan");  
console.log(myCar);
```

3. Structural Design Patterns

Adapter Pattern

Converts one interface to another.

typescript

```
class OldSystem {  
  request(): string {  
    return "Old system response";  
  }  
}
```

```
class NewSystem {  
  specificRequest(): string {  
    return "New system response";  
  }  
}
```

```
class Adapter extends OldSystem {  
  private newSystem: NewSystem;
```

```
  constructor(newSystem: NewSystem) {  
    super();  
    this.newSystem = newSystem;  
  }
```

```
  request(): string {  
    return this.newSystem.specificRequest();  
  }
```

```
}  
}
```

```
const adapted = new Adapter(new NewSystem());  
console.log(adapted.request()); // "New system response"
```

4. Behavioral Design Patterns

Observer Pattern

Allows objects to subscribe and get notified of changes.

typescript

```
class Observer {  
  update(data: any) {  
    console.log("Observer received:", data);  
  }  
}
```

```
class Subject {  
  private observers: Observer[] = [];  
  
  addObserver(observer: Observer) {  
    this.observers.push(observer);  
  }
```

```
  notify(data: any) {  
    this.observers.forEach(observer => observer.update(data));  
  }  
}
```

```
const subject = new Subject();  
const observer1 = new Observer();  
const observer2 = new Observer();
```

```
subject.addObserver(observer1);  
subject.addObserver(observer2);  
subject.notify("Hello Observers!");
```

5. Best Practices for TypeScript Design Patterns

- Use strict types to improve code safety.
- Avoid unnecessary complexity in pattern implementations.
- Use interfaces for better abstraction and maintainability.
- Optimize performance by avoiding redundant object creation.

Practice Questions:

1. What is the Singleton pattern, and why is it useful?
2. Explain the Factory pattern with an example.
3. How does the Adapter pattern help in system integration?
4. Implement an Observer pattern where multiple observers track changes in a stock price.
5. Why should we use design patterns in software development?

Practice Answers:

1. The Singleton pattern ensures a class has only one instance, preventing redundant object creation and saving memory.

2. The Factory pattern creates objects without exposing the instantiation logic. Example:

typescript

```
class Product {  
  constructor(public name: string, public price: number) {}  
}
```

```
class ProductFactory {  
  static createProduct(type: string): Product {  
    if (type === "phone") return new Product("Phone", 699);  
    if (type === "laptop") return new Product("Laptop", 1299);  
    throw new Error("Unknown product type");  
  }  
}
```

```
const phone = ProductFactory.createProduct("phone");  
console.log(phone);
```

3. The Adapter pattern converts an interface into another interface the client expects, improving compatibility.

4. Observer Pattern Example:

typescript

```
class StockObserver {  
  update(price: number) {  
    console.log("New stock price:", price);  
  }  
}
```

```
class StockMarket {  
  private observers: StockObserver[] = [];  
  private price: number = 100;
```

```
  addObserver(observer: StockObserver) {  
    this.observers.push(observer);  
  }
```

```
  setPrice(newPrice: number) {  
    this.price = newPrice;  
    this.notify();  
  }
```

```
  notify() {  
    this.observers.forEach(observer => observer.update(this.price));  
  }  
}
```

```
const market = new StockMarket();  
const investor = new StockObserver();  
market.addObserver(investor);  
market.setPrice(120);
```

5. Design patterns promote reusable, maintainable, and scalable code by providing structured solutions to common problems.